

React

Building UIs with Components

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Understand components, JSX, and props
- ▶ Manage state with `useState` and side effects with `useEffect`
- ▶ Handle forms, lists, and data fetching
- ▶ Split components, extract custom hooks, and use refs
- ▶ Use React DevTools to inspect and debug

Section Items

- ▶ Setting Up with Vite
- ▶ JSX and Props
- ▶ useState
- ▶ Lists and Keys
- ▶ Forms
- ▶ Lifting State Up
- ▶ useEffect and localStorage
- ▶ Fetching Data
- ▶ Filters and Derived State
- ▶ Component Splitting
- ▶ Custom Hooks
- ▶ useRef and DevTools

Why React?

- ▶ The DOM is hard to keep in sync with your data
- ▶ React flips it: you **describe** what the screen should show. React **figures out** how.
- ▶ One pattern (components + state + props) scales from a toggle button to Facebook

What Is a Component?

```
function Hello() {  
  return <h1>Hello, world!</h1>;  
}
```

- ▶ A function that returns JSX
- ▶ That's the whole idea

► **Setting Up with Vite**

- JSX and Props
- useState
- Lists and Keys
- Forms
- Lifting State Up
- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Setting Up with Vite

```
npm create vite@latest  
# Pick "React" -> "TypeScript"  
npm install  
npm run dev
```

- ▶ Scaffolds a new React + TypeScript project
- ▶ Hot reload — save and see changes instantly

The Files We Care About

- ▶ `index.html` — almost empty, just a `<div id="root">`
- ▶ `src/main.tsx` — entry point, hands the page to React
- ▶ `src/App.tsx` — your first component
- ▶ Everything else is config — ignore it for now

JSX Is JavaScript

- ▶ Looks like HTML, but it's JavaScript
- ▶ `<h1>Hello</h1>` compiles to `React.createElement("h1", null, "Hello")`
- ▶ Can't use `class` (reserved word) → use `className`
- ▶ Expressions go inside `{}`: `{name}`, `{2 + 2}`, `{items.length}`

- ▶ Open `src/App.tsx`, show the `<h1>`
- ▶ Change the text, save, watch the browser update instantly

Section Items

- Setting Up with Vite

► **JSX and Props**

- useState
- Lists and Keys
- Forms
- Lifting State Up

- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Components Take Props

```
function TodoCard({ todo }: { todo: Todo }) {  
  return <li>{todo.text}</li>;  
}
```

- ▶ Props = data flowing IN to a component
- ▶ Destructured from a single argument object
- ▶ TypeScript types catch wrong props at compile time

Passing Props

```
<TodoCard todo={todo1} />
```

- ▶ Like HTML attributes, but values are JavaScript inside `{}`
- ▶ Same Todo shape as in our TS demo: `{ id, text, done, minutes }`

One Component, Many Uses

- ▶ Same `TodoCard` renders three different todos
- ▶ That's the reuse win — write once, render anywhere
- ▶ Change the component → all instances update

- ▶ Add a 4th todo
- ▶ Render with `<TodoCard todo={todo4} />`

Section Items

- Setting Up with Vite
- JSX and Props
- ▶ **useState**
 - Lists and Keys
 - Forms
 - Lifting State Up
- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Interactivity Needs State

- ▶ The screen has to remember things between clicks
- ▶ React's answer: `useState`
- ▶ State is data that changes over time and triggers re-renders

The Shape of useState

```
const [value, setValue] = useState(initial);
```

- ▶ Returns an array: current value + setter function
- ▶ Array destructuring (same as `const [a, b] = [1, 2]`)
- ▶ Calling the setter triggers a re-render

NEVER Mutate State

```
// WRONG -- same reference, no re-render
todo.done = !todo.done;
setTodo(todo);

// RIGHT -- new object, re-renders
setTodo({ ...todo, done: !todo.done });
```

- ▶ React compares by reference — same object = no update
- ▶ Always create a NEW object or array

Hook Rules

- ▶ Must be called at the **top** of a component
- ▶ Same order every render
- ▶ Never inside `if` / `for` / after early return
- ▶ These rules apply to ALL hooks, not just `useState`

Live Code: snapshot-03

- ▶ Add a counter to App
- ▶ Click and watch the number go up

Section Items

- Setting Up with Vite
- JSX and Props
- useState
- ▶ **Lists and Keys**
- Forms
- Lifting State Up
- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Rendering Arrays

```
{todos.map((todo) => (  
  <TodoCard key={todo.id} todo={todo} />  
))}
```

- ▶ `.map()` produces an array of JSX → React renders them
- ▶ Most common pattern in any React app

The key Prop

- ▶ React uses keys to track items across renders
- ▶ Without keys → “key” warning in console + subtle bugs
- ▶ **Don't use array index as key** — use a stable, unique ID
- ▶ Keys must be unique among siblings

Updating Arrays Immutably

Action	Pattern
Add	<code>setTodos([...todos, x])</code>
Remove	<code>setTodos(todos.filter(t => t.id !== id))</code>
Update one	<code>setTodos(todos.map(t => t.id === id ? {...t, done: true} : t))</code>

- ▶ Delete the `key=` and show the console warning
- ▶ Add it back — warning disappears

Break

10 minutes

Section Items

- Setting Up with Vite
- JSX and Props
- useState
- Lists and Keys
- **Forms**
 - Lifting State Up
 - useEffect and localStorage
 - Fetching Data
 - Filters and Derived State
 - Component Splitting
 - Custom Hooks
 - useRef and DevTools

Controlled Inputs

```
<input  
  value={text}  
  onChange={(e) => setText(e.target.value)}  
>
```

- ▶ State is the source of truth
- ▶ Every keystroke fires `onChange` → updates state → updates the input

Why Controlled Inputs?

- ▶ Validate as the user types
- ▶ Format or mask input
- ▶ Show a live preview
- ▶ Disable submit when empty

Form Submission

```
<form onSubmit={(e) => {  
  e.preventDefault();  
  addTodo(text, minutes);  
  setText("");  
}}>
```

- ▶ Use `<form onSubmit>`, not `<button onClick>`
- ▶ Always `event.preventDefault()` first
- ▶ Submit-on-Enter works automatically with `<form>`

- ▶ Type in the input, see the preview update on every keystroke
- ▶ Submit the form, watch the todo appear

Section Items

- Setting Up with Vite
- JSX and Props
- useState
- Lists and Keys
- Forms
- ▶ **Lifting State Up**
- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Where Should State Live?

- ▶ “In the lowest component that needs it”
- ▶ If two siblings need the same state → lift to their parent
- ▶ State flows DOWN via props, events flow UP via callbacks

Data Down, Callbacks Up

```
<TodoCard todo={todo} onToggle={toggleTodo} />
```

- ▶ Parent passes data AND a callback
- ▶ Child calls the callback when something happens
- ▶ Parent decides what the callback does

Updating One Item in an Array

```
todos.map((t) =>
  t.id === id ? { ...t, done: !t.done } : t
)
```

- ▶ Map over the array
- ▶ Replace the matching item with a NEW object
- ▶ Leave everything else unchanged

- ▶ Click checkbox, click delete
- ▶ Show the parent's state changing in React DevTools

Section Items

- Setting Up with Vite
- JSX and Props
- useState
- Lists and Keys
- Forms
- Lifting State Up
- ▶ **useEffect and localStorage**
 - Fetching Data
 - Filters and Derived State
 - Component Splitting
 - Custom Hooks
 - useRef and DevTools

Side Effects

- ▶ Some code shouldn't run during render
- ▶ Examples: `fetch`, `timers`, `localStorage`, `subscriptions`
- ▶ React's answer: `useEffect`

The Shape of useEffect

```
useEffect(() => {  
  // side effect code here  
}, [dependencies]);
```

Dependencies	When it runs
[]	Once, on mount
[x]	On mount, then when x changes
(none)	After every render — usually a bug

Persisting State

- ▶ Read once on mount → lazy initializer in `useState`
- ▶ Write on every change → `useEffect` with `[todos]` dependency
- ▶ `JSON.stringify` to save, `JSON.parse` to load (same as vanilla JS)

StrictMode Runs Effects Twice

- ▶ In development only — intentional, surfaces bugs
- ▶ Production runs them once
- ▶ If your effect breaks when run twice, it has a bug

- ▶ Add todos, refresh the page, watch them persist
- ▶ Open DevTools → Application → Local Storage

Section Items

- Setting Up with Vite
- JSX and Props
- useState
- Lists and Keys
- Forms
- Lifting State Up
- useEffect and localStorage
- ▶ **Fetching Data**
 - Filters and Derived State
 - Component Splitting
 - Custom Hooks
 - useRef and DevTools

Fetch Goes in useEffect

```
useEffect(() => {  
  async function load() {  
    const res = await fetch(url);  
    const data = await res.json();  
    setTodos(data);  
  }  
  load();  
}, []);
```

- ▶ Can't make useEffect's callback itself async
- ▶ Define inner async function and call it

Three UI States

Every fetch has three possible UI states — handle all of them:

State	Show
Loading	Spinner or skeleton
Error	Error message
Data	The actual UI

Map API Shape → App Shape

```
const mapped: Todo[] = data.map((t) => ({
  id: t.id,
  text: t.title,
  done: t.completed,
  minutes: 15,
})));
```

- ▶ Don't let an API dictate your component types
- ▶ Map at the boundary, use your own types everywhere else

- ▶ Open Network tab, refresh, show the fetch + the response

Coming Soon

- ▶ TanStack Query handles loading, error, caching, and refetching for you
- ▶ We'll see it next class — you're learning the raw version first

End of Class 4

Next: Filtering, Splitting, Hooks, DevTools